

Introduction to Object-Oriented Software Construction

Experiment Guidebook

Experiment 2: Factory Pattern

Harbin Institute of Technology (Shenzhen)

2025

1. Objective

- (1) Understand the motivation and design intent of the Factory Pattern
- (2) Master the structure of the Factory Pattern and be able to draw its UML class diagram
- (3) Refactor existing code and implement the Factory Pattern in practice

2. Environment

- Operating System: Windows 10
- IDE: IntelliJ IDEA 2025.2
- JDK: OpenJDK 24 or higher
- Tools: PlantUML plugin for IntelliJ IDEA

3. Experiment Tasks

- (1) **Iterative Development Goal:** Add a Boss Enemy Aircraft to the game

Specific requirements are as follows:

Boss Enemy Aircraft	
Appearance	Appears when score reaches threshold, multiple times
Movement	overs and moves left-right above screen
Firepower	Circle fire trajectory, 20 bullets at once
Destroyed	Randomly drops ≤ 2 power-up supplies
	

- (2) **Design Pattern Application:** Identify scenarios suitable for the **Factory Pattern**, draw the UML class diagram and then refactor the codebase to apply it to the creation of both enemy aircraft and power-up supplies, thereby enhancing extensibility and maintainability.

4. Experiment Steps

4.1 Add Boss Enemy Aircraft

Refactor the existing codebase to introduce a new enemy type — the Boss Enemy. **Create a new `BossEnemy` class** that extends the `AbstractEnemyAircraft` class, and implement its unique behaviors such as power-up supply generation, firing trajectory, and controlled spawning logic (managed by the `Game` class).

Tasks to be completed:

The version submitted in this experiment must include and correctly implement the following functions:

- The boss enemy appears whenever the player's score reaches a threshold (`bossScoreThreshold = 300`).
- It can reappear multiple times if the threshold is met again, but only one boss can be on the screen at a time.
- It hovers near the top of the screen and moves left and right continuously.
- It attacks by firing 20 bullets simultaneously in a circular (ring-shaped) trajectory.
- When destroyed, it randomly drops 0, 1, or 2 power-up supplies (with a maximum of 2).

Reference code for circular fire trajectory:

```
public List<BaseBullet> shoot() {
    List<BaseBullet> res = new LinkedList<>();

    //the x-coordinate and y-coordinate of boss
    int x = this.getLocationX();
    int y = this.getLocationY();

    int bulletSpeed = 10;
    int bulletRadius = 100;
    int angleIncrement = 360 / shootNum;

    BaseBullet bullet;
    for (int i = 0; i < shootNum; i++) {
        int angle = i * angleIncrement;
        int bulletX = (int) (x + bulletRadius * Math.cos(Math.toRadians(angle)));
        int bulletY = (int) (y + bulletRadius * Math.sin(Math.toRadians(angle)));
        int bulletSpeedX = (int) (bulletSpeed * Math.cos(Math.toRadians(angle)));
        int bulletSpeedY = (int) (bulletSpeed * Math.sin(Math.toRadians(angle)));
        bullet = new EnemyBullet(bulletX, bulletY, bulletSpeedX, bulletSpeedY, power);
        res.add(bullet);
    }
}
```

```

    return res;
}

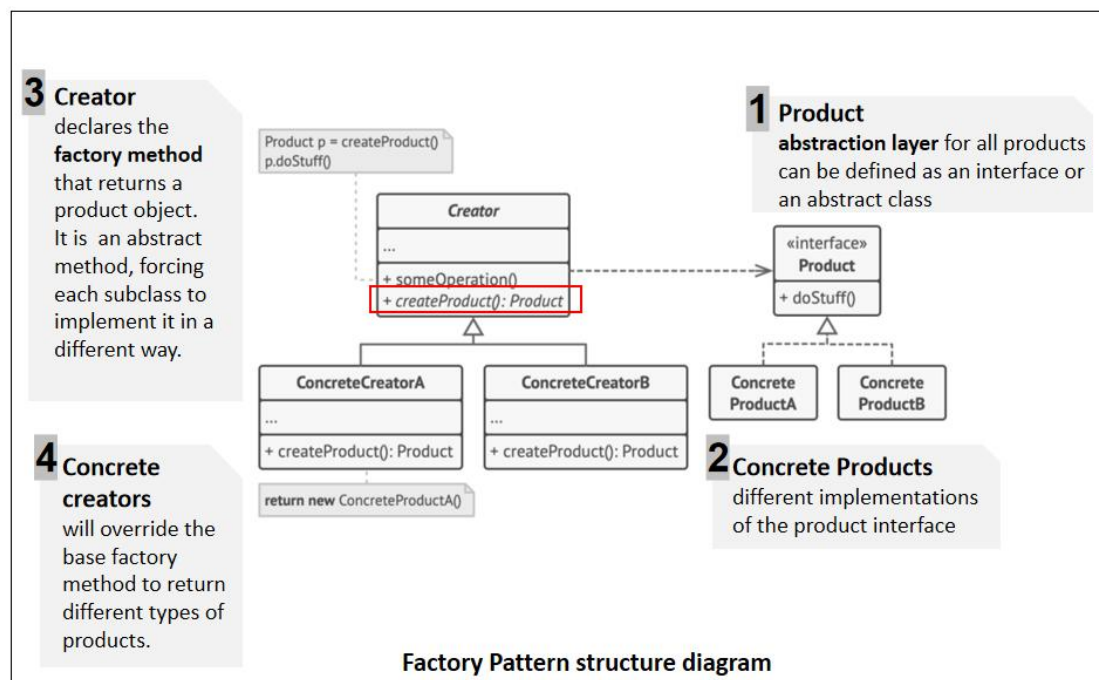
```

4.2 Factory Pattern

The **Factory Pattern** is a creational design pattern that provides a method in a parent class for creating objects, allowing subclasses to determine the specific type of object to instantiate.

Key points of the Factory Pattern:

- ✧ **Decouples Object Creation:** Separates the process of object instantiation from the client code that uses the objects, reducing direct dependencies between classes.
- ✧ **Relies on Abstraction:** Uses abstract product interfaces/classes to define the common behavior of products, and abstract factory interfaces/classes to declare object creation methods.
- ✧ **Concrete Implementations:** Concrete product classes implement the abstract product interface, while concrete factory classes override the factory method to create specific product instances.
- ✧ **Promotes Extensibility:** Adding new product types only requires creating corresponding concrete product and factory classes, without modifying existing client code (follows the Open/Closed Principle).



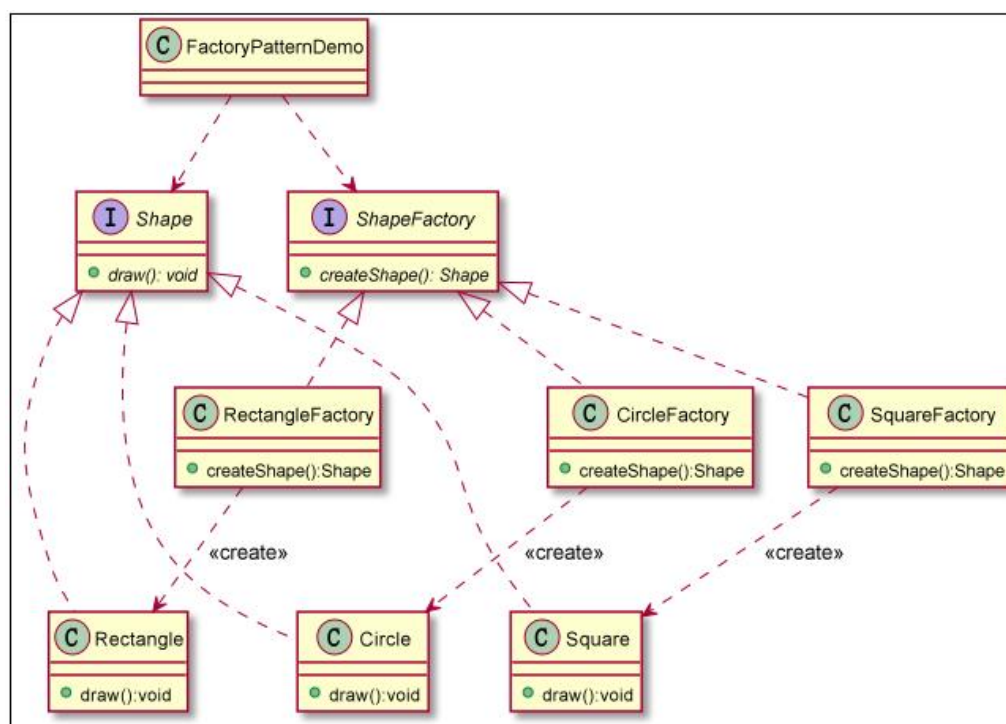
Tasks to be completed:

- (1) Please refer to the above Factory Pattern structure diagram to **draw the Factory pattern UML class diagram** for the **enemy aircrafts** in this game. [The client side does not need to be drawn](#). Save the diagram in the uml directory.
- (2) Based on your designed UML class diagram, **refactor the code** and create all enemy aircraft using the Factory Pattern, ensuring all enemy instances are created through factories.
- (3) Similarly, use PlantUML to draw a class diagram applying the Factory Pattern to **power-up supplies**.
- (4) **Refactor** the power-up supply creation logic, ensuring all power-up instances are created through factories.

4.3 Factory Pattern Application Example

Suppose we want to develop a graphics factory that can produce three types of graphic products: circles, rectangles, and squares. How should we implement this using the Factory Pattern?

The following is the UML class diagram design for applying the Factory Pattern to our graphics factory problem.



Code examples of the Factory pattern :

First, we define a Shape interface and concrete classes that implement it. Next, we create a ShapeFactory class to encapsulate object creation logic. Finally, the client class FactoryPatternDemo uses ShapeFactory to obtain different Shape instances without directly instantiating them.

Step 1: Create an interface to act as the Product role

Shape.java

```
public interface Shape {  
    void draw();  
}
```

Note: Shape can also be implemented as an abstract class. In code implementation, the abstraction layer can be realized using either an interface or an abstract class.

Step 2: Create entity classes that implement the interface, acting as Concrete Product roles

Rectangle.java

```
public class Rectangle implements Shape {  
  
    @Override  
    public void draw() {  
        System.out.println("Inside Rectangle::draw() method.");  
    }  
}
```

Square.java

```
public class Square implements Shape {  
  
    @Override  
    public void draw() {  
        System.out.println("Inside Square::draw() method.");  
    }  
}
```

Circle.java

```
public class Circle implements Shape {  
  
    @Override  
    public void draw() {  
        System.out.println("Inside Circle::draw() method.");  
    }  
}
```

```
}
```

Step 3: Create a factory interface to act as the Creator role

ShapeFactory.java

```
public interface ShapeFactory {  
  
    public abstract Shape createShape();  
  
}
```

Step 4: Create concrete factory classes that implement the factory interface, acting as Concrete Creator roles

RectangleFactory.java

```
public class RectangleFactory implements ShapeFactory {  
  
    @Override  
    public Shape createShape() {  
        return new Rectangle();  
    }  
  
}
```

SquareFactory.java

```
public class SquareFactory implements ShapeFactory {  
  
    @Override  
    public Shape createShape() {  
        return new Square();  
    }  
  
}
```

CircleFactory.java

```
public class CircleFactory implements ShapeFactory {  
  
    @Override  
    public Shape createShape() {  
        return new Circle();  
    }  
  
}
```

Step 5: Use FactoryPatternDemo to demonstrate the usage of the Factory Pattern

FactoryPatternDemo.java

```
public class FactoryPatternDemo {
```

```
public static void main(String[] args) {  
  
    ShapeFactory shapeFactory ;  
    Shape shape;  
  
    //Obtain the Circle object and call its draw method  
    shapeFactory = new CircleFactory();  
    shape = shapeFactory.createShape();  
    shape.draw();  
  
    //Obtain the Rectangle object and call its draw method  
    shapeFactory = new RectangleFactory();  
    shape = shapeFactory.createShape();  
    shape.draw();  
  
    //Obtain the Square object and call its draw method  
    shapeFactory = new SquareFactory();  
    shape = shapeFactory.createShape();  
    shape.draw();  
  
    }  
}
```

Step 6: Execute the program and output the results

```
Inside Circle::draw() method.  
Inside Rectangle::draw() method.  
Inside Square::draw() method.
```

5. Assignment Submission

Submission requirements include:

(1) Project Code (the entire project folder "AircraftWar-base")

- UML class diagrams drawn using the PlantUML plugin (saved in the uml directory)
- Fully functional and runnable code

(2) Experiment Report

Please use the provided report template

Place both the project code and the lab report in the same folder, then compress the entire folder into a single archive file (.zip) for submission.

Q

Deadline:

Submit your work to the **HITsz Grader** assignment submission platform **within one**

week after the lab session. The exact due date will be specified on the platform.

Login URL: <https://grader.cs-lab.top/>

The default username and password are your student ID.

Note: After uploading, please download your submission to verify that it was uploaded successfully.